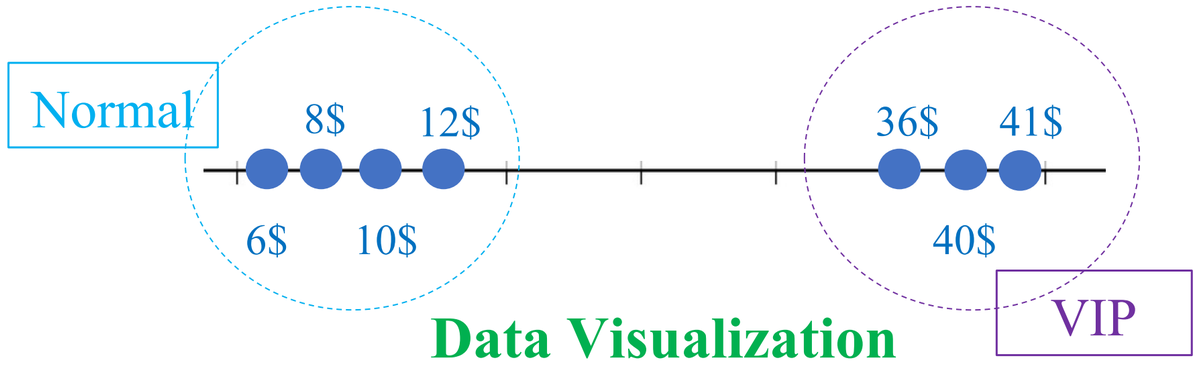
UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Môn Máy Học
thành viên nhóm 3

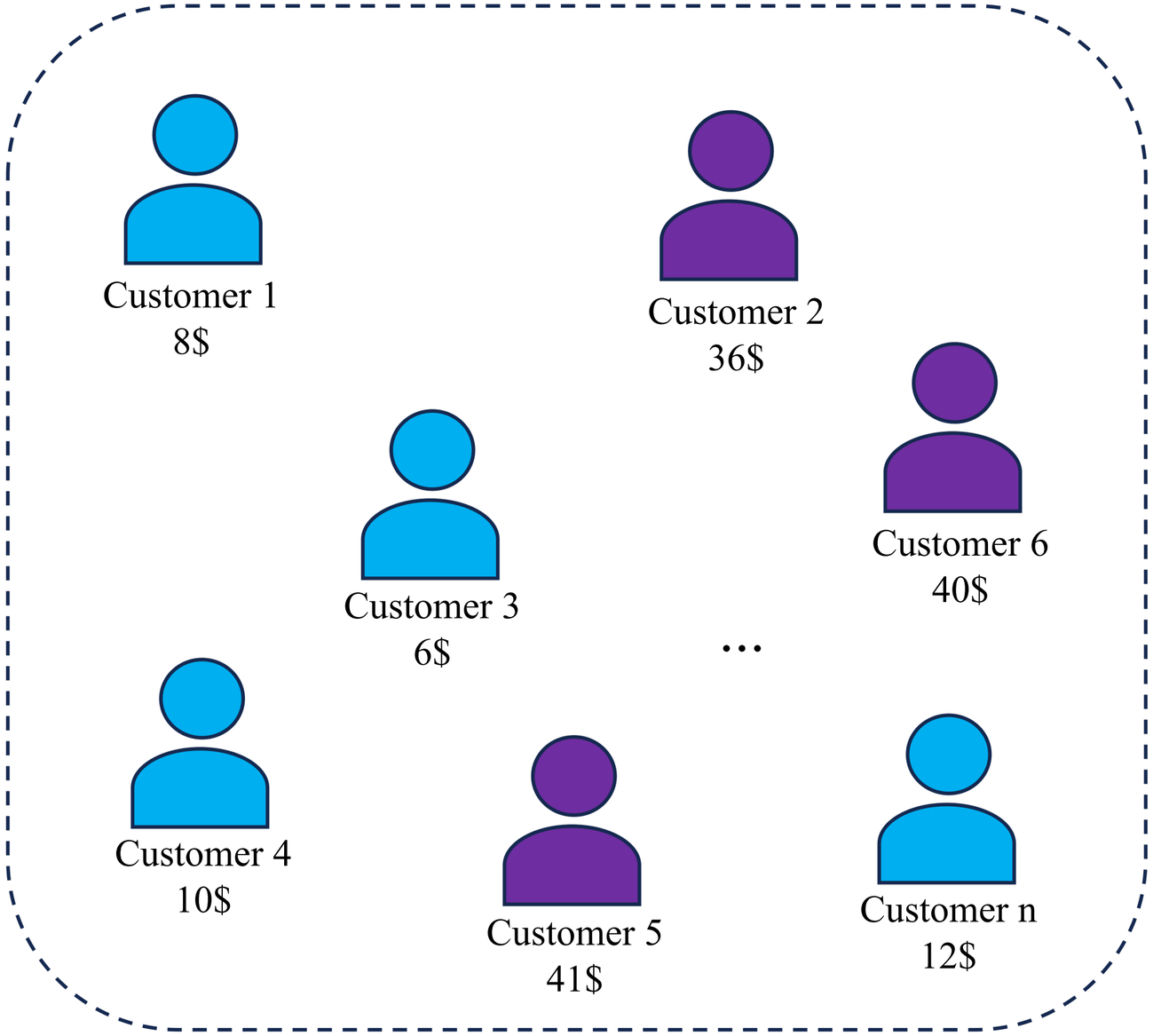
Họ và Tên	MSSV	Chức vụ
Lê Chí Bảo	051206002212	Nhóm trưởng
Nguyễn Quốc Dung Bình	058306000802	Thành viên
Nguyễn Đặng Mỹ Anh	051306002813	Thành viên
Ôn Thành Công	089206018985	Thành viên
Nguyễn Hải Anh	036206006705	Thành viên

K-Means Clustering là gì

- Thuật toán học không giám sát (Unsupervised Learning)
- Dùng để phân cụm dữ liệu (Clustering)
- Input: dữ liệu chưa có nhãn
- Output: chia thành K nhóm (clusters)



Data Visualization



Ý tưởng hoạt động của K-Means

- Chọn trước số cụm k
- Khởi tạo ngẫu nhiên k centroid ban đầu
- Tính khoảng cách từ mỗi điểm dữ liệu đến các centroid
- Gán mỗi điểm vào cụm có centroid gần nhất
- Cập nhật centroid bằng trung bình các điểm trong cụm
- Lặp lại hai bước gán cụm và cập nhật tâm cụm cho đến khi hội tụ

1) A data point will belong to Set_0 if it is closer to C_0

$$Set_0 = \{x_p : |x_p - C_0| \leq |x_p - C_1|\}$$

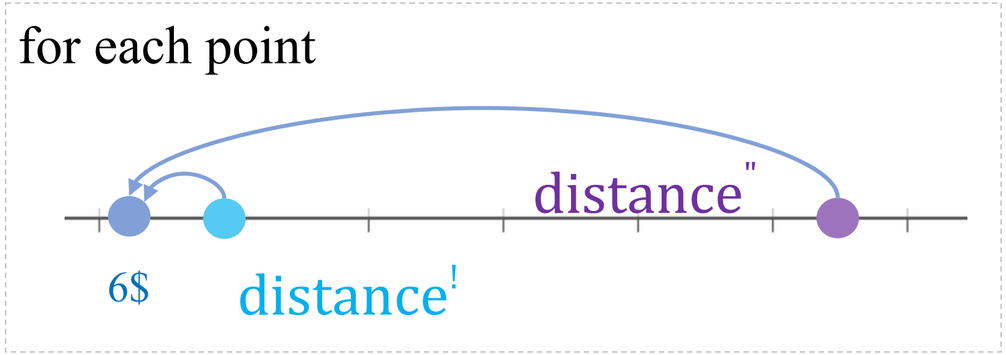
$\Rightarrow$ can compute R_0

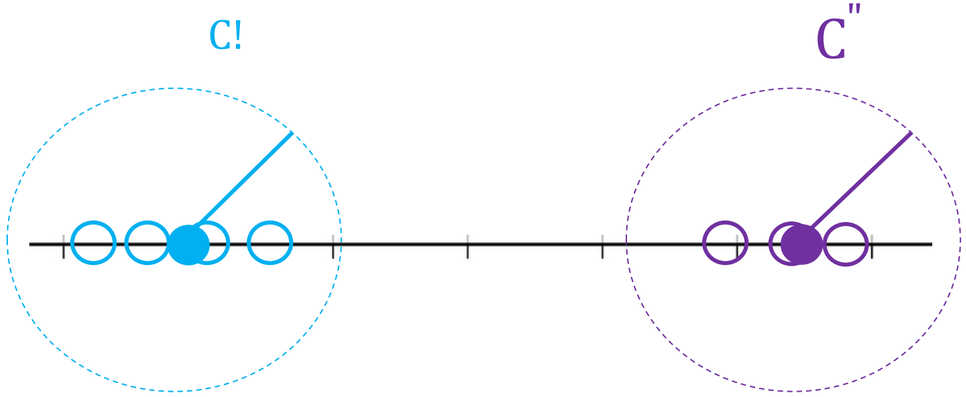
2) A data point will belong to Set_1 if it is closer to C_1

$$Set_1 = \{x_p : |x_p - C_1| < |x_p - C_0|\}$$

$\Rightarrow$ can compute R_1

- K-Means chia dữ liệu thành k cụm sao cho các điểm trong cùng một cụm giống nhau hơn và gần tâm cụm hơn.





Công thức và điều kiện dừng

Assignment step

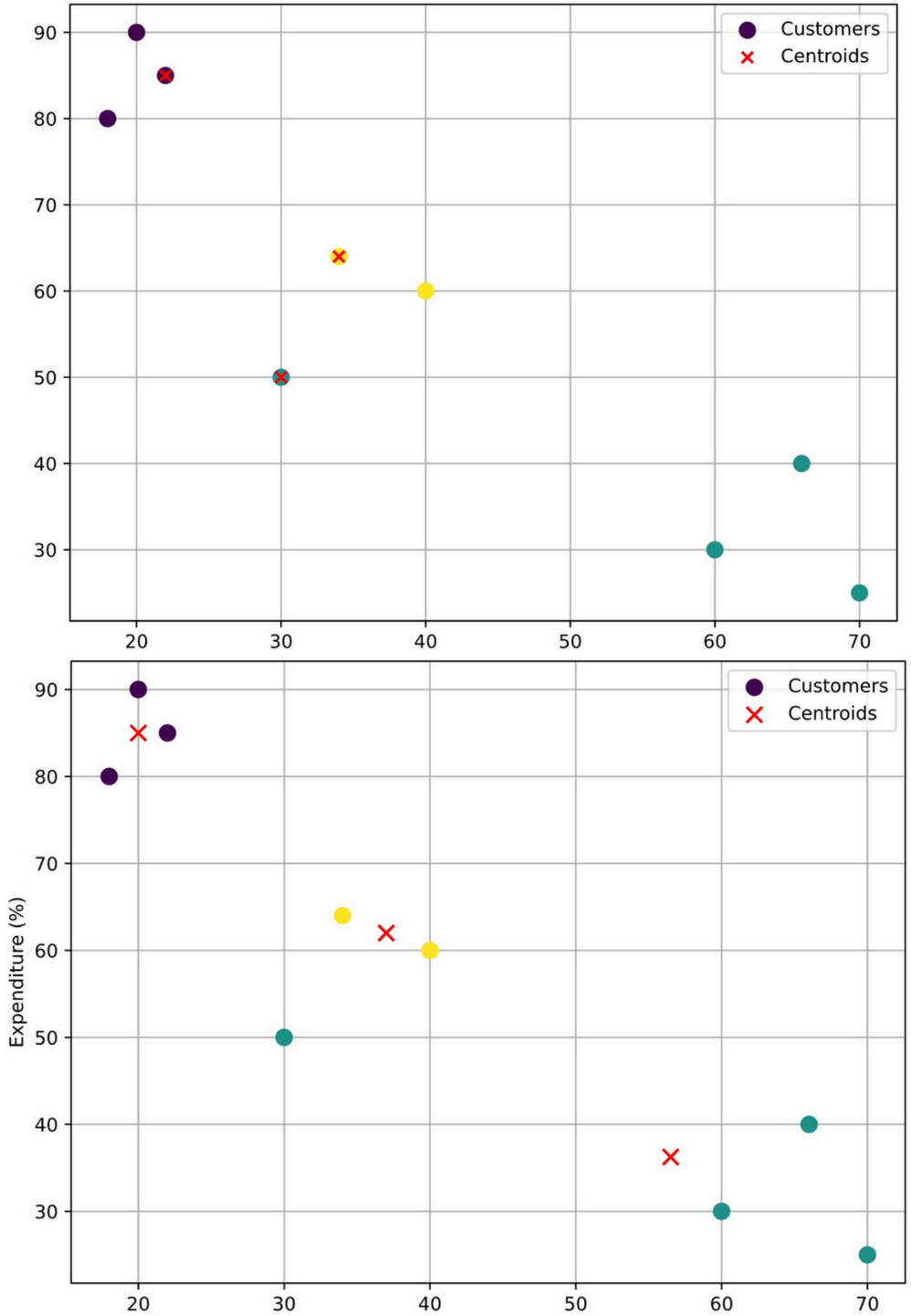
$$S_i^{(t)} = \left\{ x_p : \left\| x_p - c_i^{(t)} \right\|^2 \le \left\| x_p - c_j^{(t)} \right\|^2 \forall j, 0 \le j < k \right\}$$

Update step

$$c_i^{(t+1)} = \frac{1}{\left| S_i^{(t)} \right|} \sum_{x \in S_i^{(t)}} x$$

Thuật toán dừng khi:

- centroid không thay đổi hoặc thay đổi rất nhỏ
- hoặc các điểm không đổi cụm
- hoặc đạt số vòng lặp tối đa



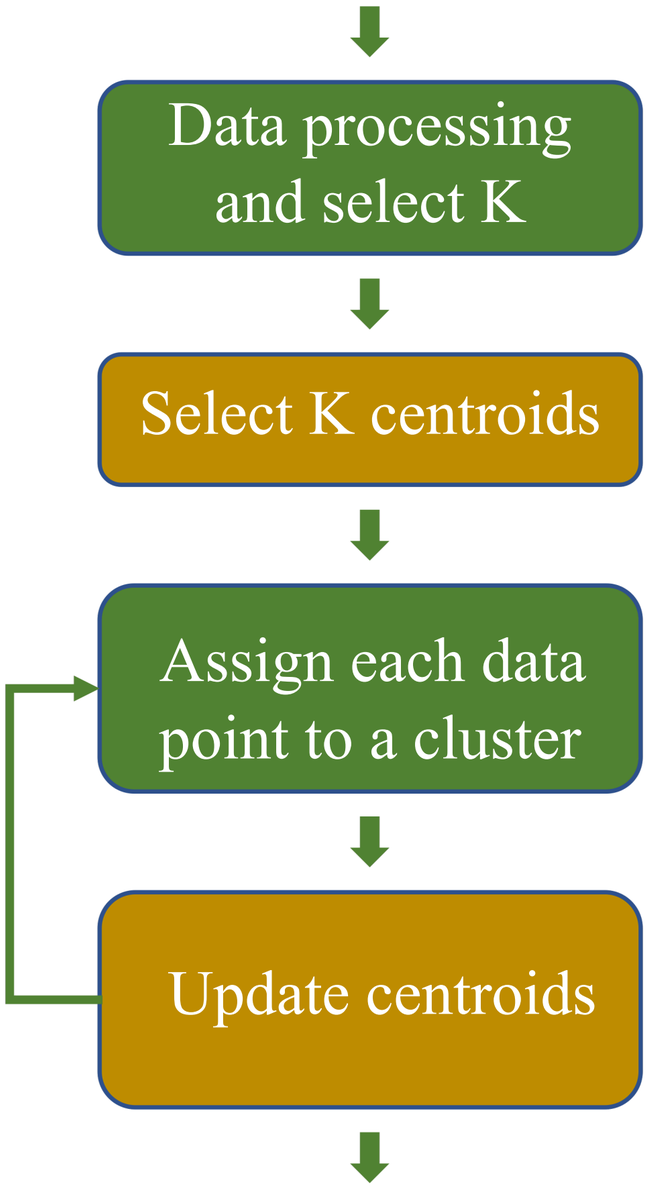
Quy trình K-Means

1. Chọn số cụm K
2. Khởi tạo K centroid (random)
3. Gán mỗi điểm vào cụm gần nhất
4. Cập nhật centroid
5. Lặp lại đến khi hội tụ

→ K-Means lặp để tối thiểu hóa khoảng cách giữa điểm dữ liệu và tâm cụm.

Thuật toán cố gắng làm WCSS nhỏ nhất

$$WCSS = \sum_{i=1}^{k} \sum_{x \in C_i} ||x - c_i||^2$$



Ưu và nhược điểm của K_MEAN

Ưu điểm

- Đơn giản và dễ hiểu
- Tốc độ xử lý nhanh
- Hiệu quả với các cụm hình cầu
- Nếu dữ liệu có dạng cluster tròn hoặc đồng nhất, K-Means hoạt động rất tốt.
- Ứng dụng đa dạng

Nhược điểm

- Phải xác định trước số cụm K
- Nhạy cảm với điểm ngoại lai
- Chỉ hoạt động tốt với các cụm hình cầu
- Khởi tạo ngẫu nhiên có thể dẫn đến kết quả khác nhau

Giới thiệu đề tài

Đề tài: Ứng dụng K-Means trong phân cụm khách hàng

- Dữ liệu khách hàng ngày càng lớn và đa dạng
- Doanh nghiệp cần hiểu hành vi tiêu dùng

→ Phân cụm giúp tìm ra các nhóm khách hàng

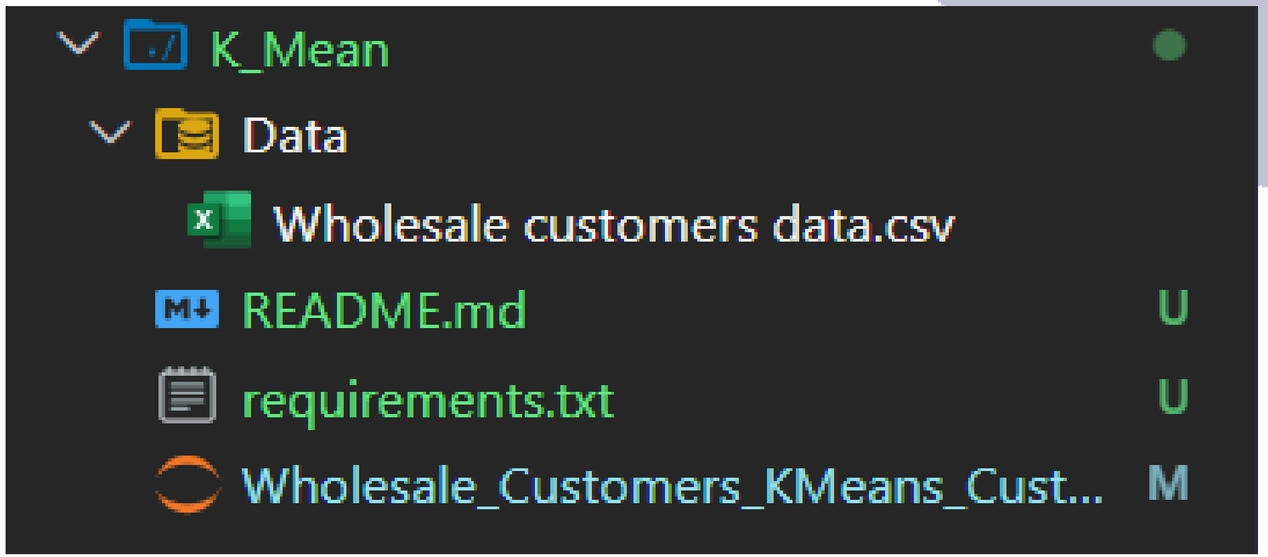
Giải pháp:

- Sử dụng thuật toán K-Means để phân nhóm khách hàng theo hành vi chi tiêu

Cấu trúc project / các file chính

Cấu trúc phần code gồm:

- File dữ liệu dataset
- File xử lí dữ liệu và huấn luyện mô hình
- File requirements



Tổng quan dataset (Wholesale Customers)

Bộ dữ liệu có các cột sau:

- Channel: loại khách hàng
- Region: khu vực
- Fresh ,Milk, Grocery, Frozen, Detergents_Paper, Delicassen: mức chi tiêu theo nhóm hàng

Trong bài toán K-means, ta thường ưu tiên các đặc trưng hành vi chi tiêu liên tục.
Hai cột Channel và Region là biến mã hóa dạng phân loại nên sẽ được xử lý riêng.

```
Channel,Region,Fresh,Milk,Grocery,Frozen,Detergents_P
2,3,12669,9656,7561,214,2674,1338
2,3,7057,9810,9568,1762,3293,1776
2,3,6353,8808,7684,2405,3516,7844
1,3,13265,1196,4221,6404,507,1788
2,3,22615,5410,7198,3915,1777,5185
2,3,9413,8259,5126,666,1795,1451
2,3,12126,3199,6975,480,3140,545
2,3,7579,4956,9426,1669,3321,2566
```

```
df = pd.read_csv("Data/Wholesale customers data.csv")
```
✓ 0.0s

```
df.head()
```
✓ 0.0s

	Channel	Region	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
0	2	3	12669	9656	7561	214	2674	1338
1	2	3	7057	9810	9568	1762	3293	1776
2	2	3	6353	8808	7684	2405	3516	7844
3	1	3	13265	1196	4221	6404	507	1788
4	2	3	22615	5410	7198	3915	1777	5185

Khám phá dữ liệu ban đầu

→ Đây là dataset về chi tiêu của 440 khách hàng, gồm các nhóm sản phẩm khác nhau. Dữ liệu không có giá trị thiếu nhưng có độ lệch lớn, do đó cần chuẩn hóa trước khi áp dụng K-Means để phân cụm khách hàng theo hành vi tiêu dùng.

```
Thông tin dữ liệu:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 440 entries, 0 to 439
Data columns (total 8 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Channel           440 non-null    int64
 1   Region            440 non-null    int64
 2   Fresh             440 non-null    int64
 3   Milk              440 non-null    int64
 4   Grocery           440 non-null    int64
 5   Frozen            440 non-null    int64
 6   Detergents_Paper  440 non-null    int64
 7   Delicassen        440 non-null    int64
dtypes: int64(8)
memory usage: 27.6 KB
```

	count	mean	std	min	25%	50%	75%	max
Channel	440.0	1.322727	0.468052	1.0	1.00	1.0	2.00	2.0
Region	440.0	2.543182	0.774272	1.0	2.00	3.0	3.00	3.0
Fresh	440.0	12000.297727	12647.328865	3.0	3127.75	8504.0	16933.75	112151.0
Milk	440.0	5796.265909	7380.377175	55.0	1533.00	3627.0	7190.25	73498.0
Grocery	440.0	7951.277273	9503.162829	3.0	2153.00	4755.5	10655.75	92780.0
Frozen	440.0	3071.931818	4854.673333	25.0	742.25	1526.0	3554.25	60869.0
Detergents_Paper	440.0	2881.493182	4767.854448	3.0	256.75	816.5	3922.00	40827.0
Delicassen	440.0	1524.870455	2820.105937	3.0	408.25	965.5	1820.25	47943.0

Làm sạch và lựa chọn đặc trưng

- giữ lại các cột chi tiêu liên tục cho K-means
- loại Channel, Region khỏi tập đặc trưng phân cụm
- sau khi phân cụm xong có thể dùng lại Channel, Region để phân tích diễn giải cụm

```
Cột mô tả (không đưa trực tiếp vào K-means): ['Channel', 'Region']
Cột chi tiêu dùng để phân cụm: ['Fresh', 'Milk', 'Grocery', 'Frozen', 'Detergents_Paper', 'Delicassen']
```

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
0	12669	9656	7561	214	2674	1338
1	7057	9810	9568	1762	3293	1776
2	6353	8808	7684	2405	3516	7844
3	13265	1196	4221	6404	507	1788
4	22615	5410	7198	3915	1777	5185

Tiền xử lý dữ liệu

Với K-means, khoảng cách Euclid rất nhạy với:

- dữ liệu lệch mạnh
- chênh lệch thang đo giữa các đặc trưng

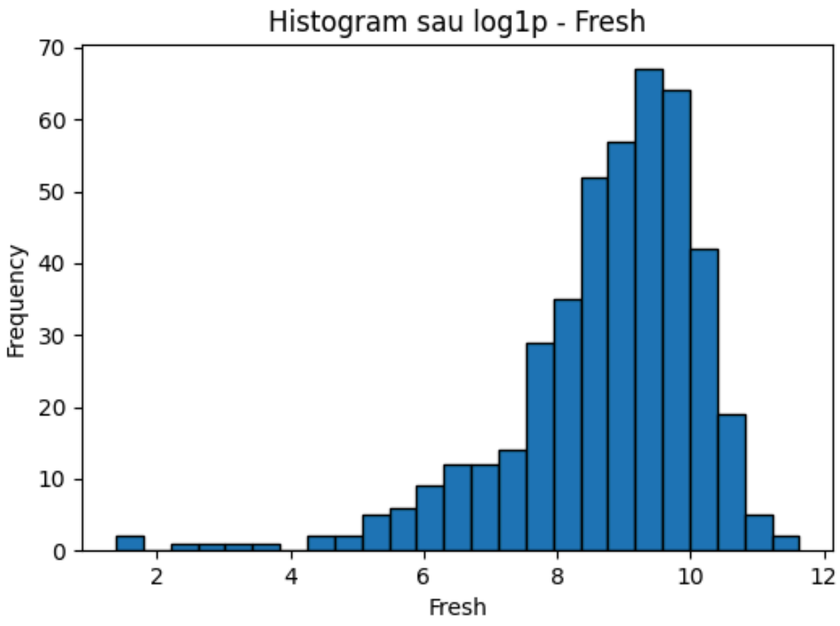
Do đó ta sẽ:

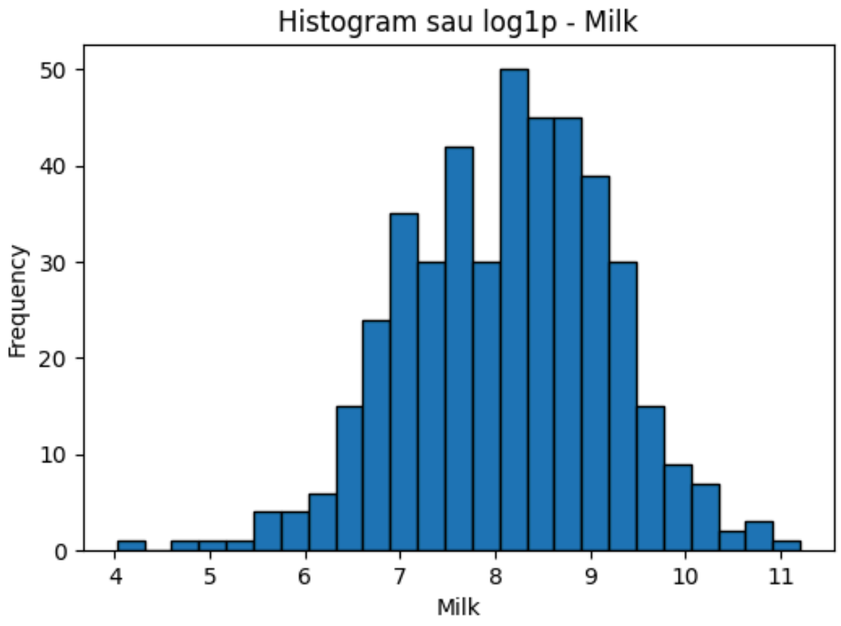
1. áp dụng log1p để giảm độ lệch
2. chuẩn hóa dữ liệu bằng z-score

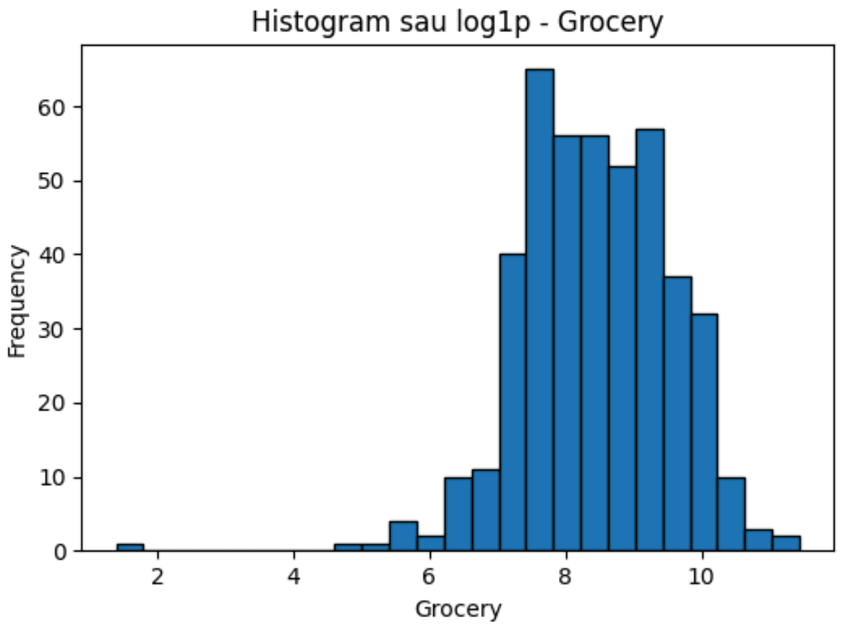
```
Shape sau tiền xử lý: (440, 6)
5 dòng đầu sau log + scale:
```

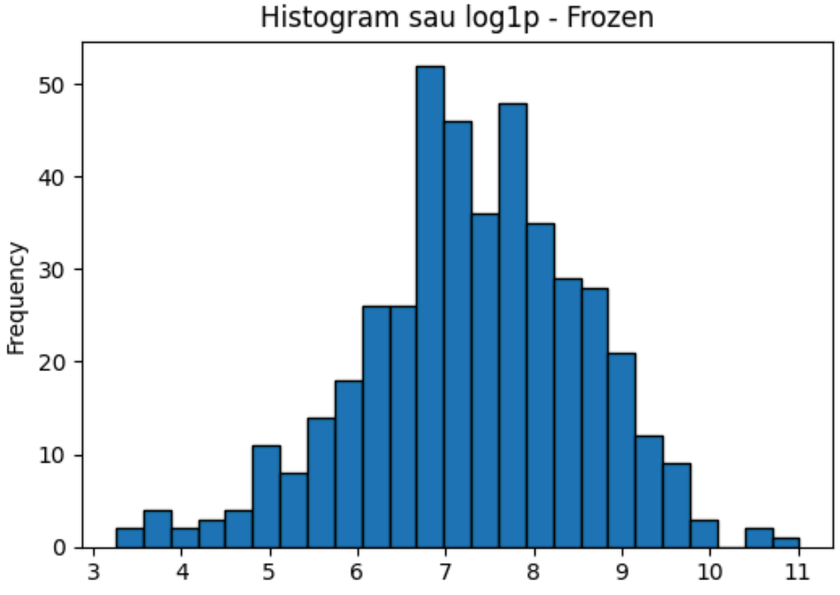
	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
0	0.486184	0.976299	0.440155	-1.509250	0.644143	0.408966
1	0.087889	0.990956	0.652171	0.134052	0.766043	0.627926
2	0.016356	0.891151	0.454687	0.376899	0.804405	1.776833
3	0.517477	-0.957973	-0.084792	1.141574	-0.328712	0.633133
4	0.880631	0.439662	0.395847	0.757322	0.404939	1.456588

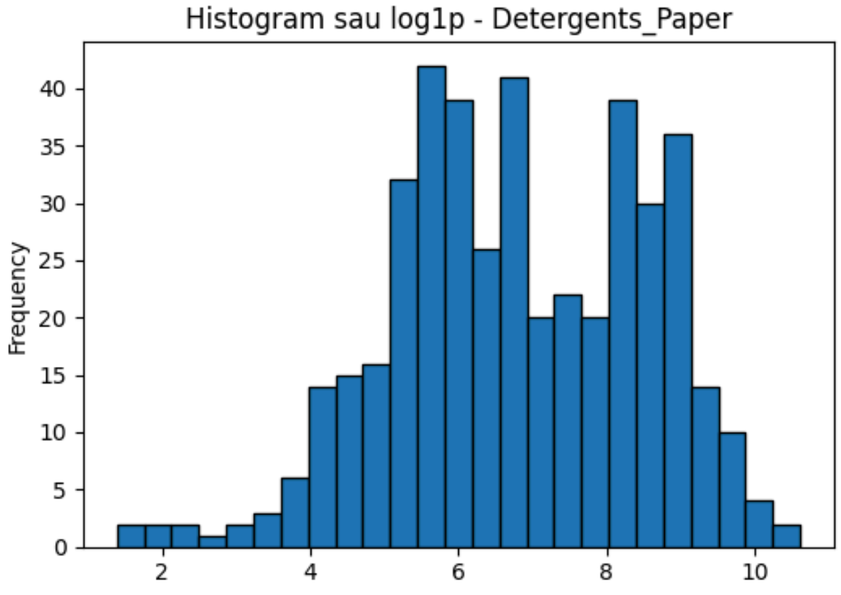
So sánh nhanh dữ liệu sau khi biến đổi log

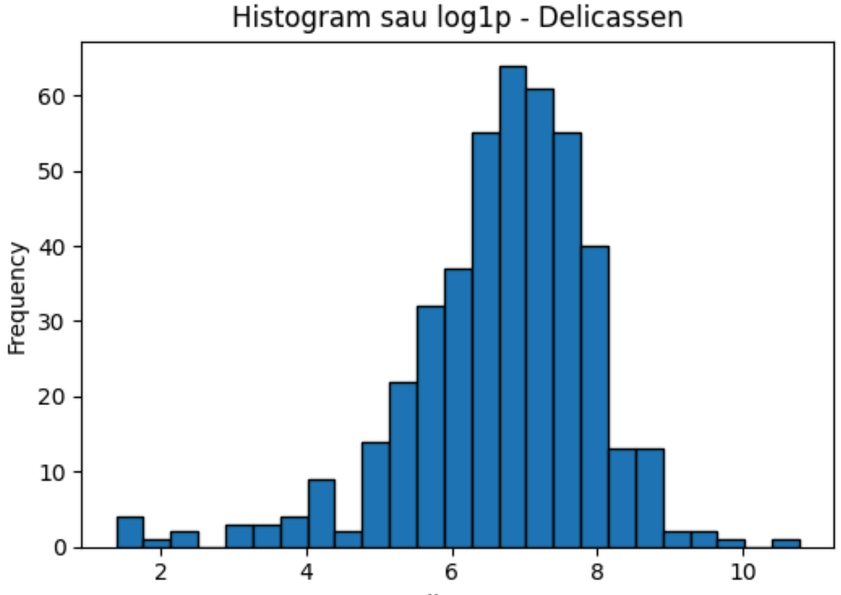












Xây dựng mô hình K-means custom

1. Các bước chính
   - Khởi tạo K centroid ngẫu nhiên
   - Tính khoảng cách từ điểm đến centroid
   - Gán điểm vào cụm gần nhất
   - Cập nhật centroid bằng trung bình
   - Lặp lại cho đến khi hội tụ

2. Các hàm chính trong code
   - _initialize_centroids() → khởi tạo tâm cụm
   - _compute_distances() → tính khoảng cách
   - _assign_labels() → gán cụm
   - _update_centroids() → cập nhật tâm cụm
   - _compute_wcss() → đánh giá chất lượng cụm

```python
class CustomKMeans:
    def __init__(self, n_clusters=3, max_iters=100, tol=1e-4, random_state=42):
        self.n_clusters = n_clusters
        self.max_iters = max_iters
        self.tol = tol
        self.random_state = random_state

        self.centroids = None
        self.labels_ = None
        self.inertia_ = None
        self.history_ = []

    def _initialize_centroids(self, X):
        rng = np.random.default_rng(self.random_state)
        indices = rng.choice(len(X), size=self.n_clusters, replace=False)
        return X[indices].copy()

    def _compute_distances(self, X, centroids):
        return np.linalg.norm(X[:, np.newaxis, :] - centroids[np.newaxis, :, :], axis=2)

    def _assign_labels(self, X, centroids):
        distances = self._compute_distances(X, centroids)
        return np.argmin(distances, axis=1)

    def _update_centroids(self, X, labels, old_centroids):
        new_centroids = []
        for i in range(self.n_clusters):
            cluster_points = X[labels == i]
            if len(cluster_points) == 0:
                new_centroids.append(old_centroids[i])
            else:
                new_centroids.append(cluster_points.mean(axis=0))
        return np.array(new_centroids)

    def _compute_wcss(self, X, labels, centroids):
        wcss = 0.0
        for i in range(self.n_clusters):
            cluster_points = X[labels == i]
            if len(cluster_points) > 0:
                wcss += np.sum((cluster_points - centroids[i]) ** 2)
        return float(wcss)

```

Quá trình huấn luyện K-Means (fit)

1. Khởi tạo
   - Chuyển dữ liệu về dạng số
   - Khởi tạo centroid ngẫu nhiên
2. Lặp tối đa max_iters

Trong mỗi vòng lặp:

   - Gán điểm vào cụm gần nhất
   - Cập nhật centroid
   - Tính độ thay đổi (shift)
   - Tính WCSS
3. Điều kiện dừng
   - Nếu centroid thay đổi rất nhỏ → dừng
4. Kết quả
   - centroids: tâm cụm cuối
   - labels: nhãn của từng điểm
   - inertia: WCSS

```python
    def fit(self, X):
        X = np.asarray(X, dtype=float)
        centroids = self._initialize_centroids(X)

        for iteration in range(self.max_iters):
            labels = self._assign_labels(X, centroids)
            new_centroids = self._update_centroids(X, labels, centroids)

            shift = np.linalg.norm(new_centroids - centroids)
            wcss = self._compute_wcss(X, labels, new_centroids)

            self.history_.append({
                "iteration": iteration + 1,
                "shift": shift,
                "wcss": wcss
            })

            if shift < self.tol:
                centroids = new_centroids
                break

            centroids = new_centroids

        self.centroids = centroids
        self.labels_ = self._assign_labels(X, self.centroids)
        self.inertia_ = self._compute_wcss(X, self.labels_, self.centroids)
        return self

    def predict(self, X):
        X = np.asarray(X, dtype=float)
        return self._assign_labels(X, self.centroids)

    def fit_predict(self, X):
        self.fit(X)
        return self.labels_
```

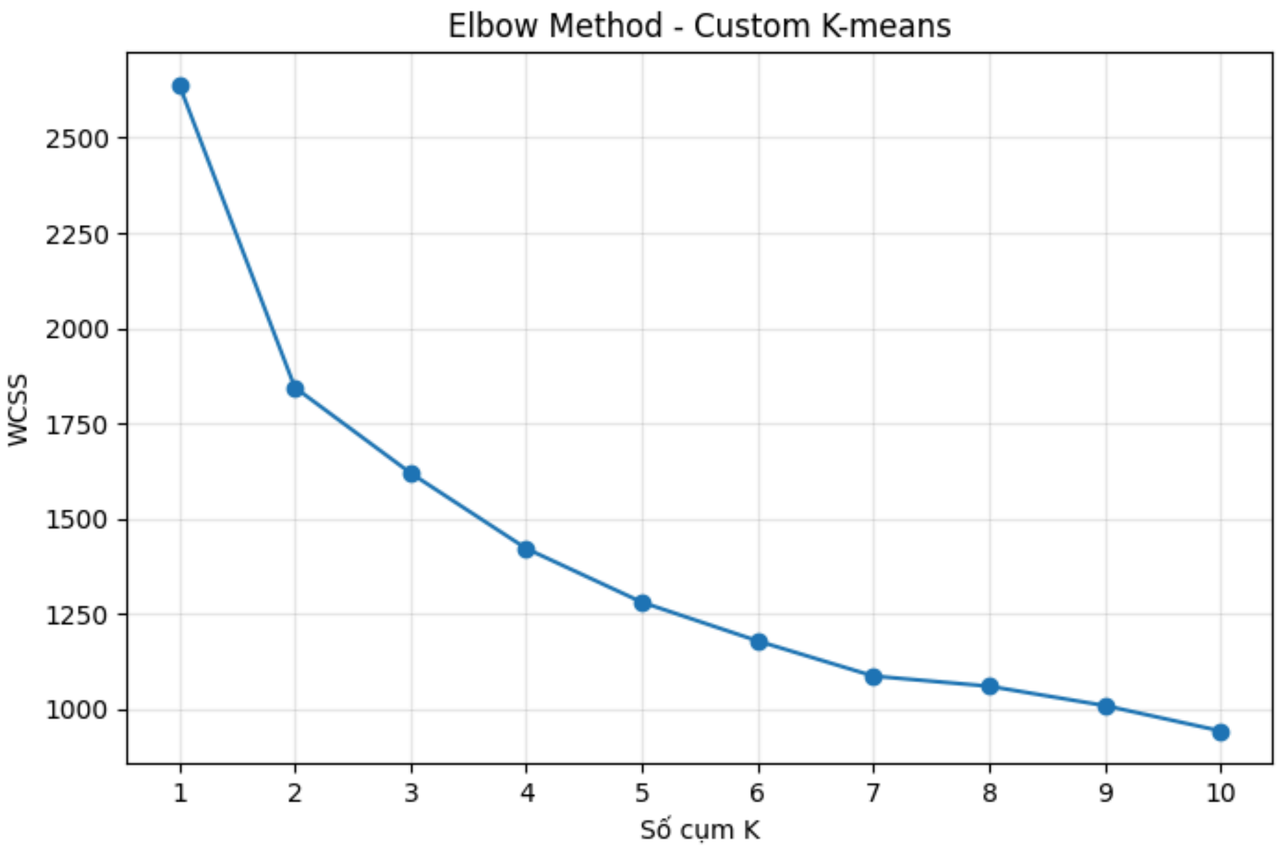
✓ 0.0s

Elbow Method để tìm K phù hợp

Từ biểu đồ Elbow Method, ta thấy WCSS giảm mạnh từ K=1 đến K=3,
sau đó tốc độ giảm chậm lại.

Điều này cho thấy số cụm tối ưu hợp lý là K=3, vì sau điểm này việc tăng số cụm không giúp cải thiện đáng kể độ chặt của cụm.

→ Do đó, ta chọn K = 3 cho mô hình K-means.



So sánh với mô hình KMeans của sklearn

→ Mặc dù kết quả giữa Custom và Sklearn không hoàn toàn giống nhau do khởi tạo ngẫu nhiên, nhưng giá trị WCSS gần tương đương và cấu trúc các cụm tương tự nhau, chứng tỏ mô hình Custom đã hoạt động đúng.

```
WCSS custom: 1619.3404248319598
WCSS sklearn: 1553.4213271044873

Số điểm trong từng cụm - custom:
0     93
1    184
2    163
Name: count, dtype: int64

Số điểm trong từng cụm - sklearn:
0     80
1    147
2    213
Name: count, dtype: int64
```

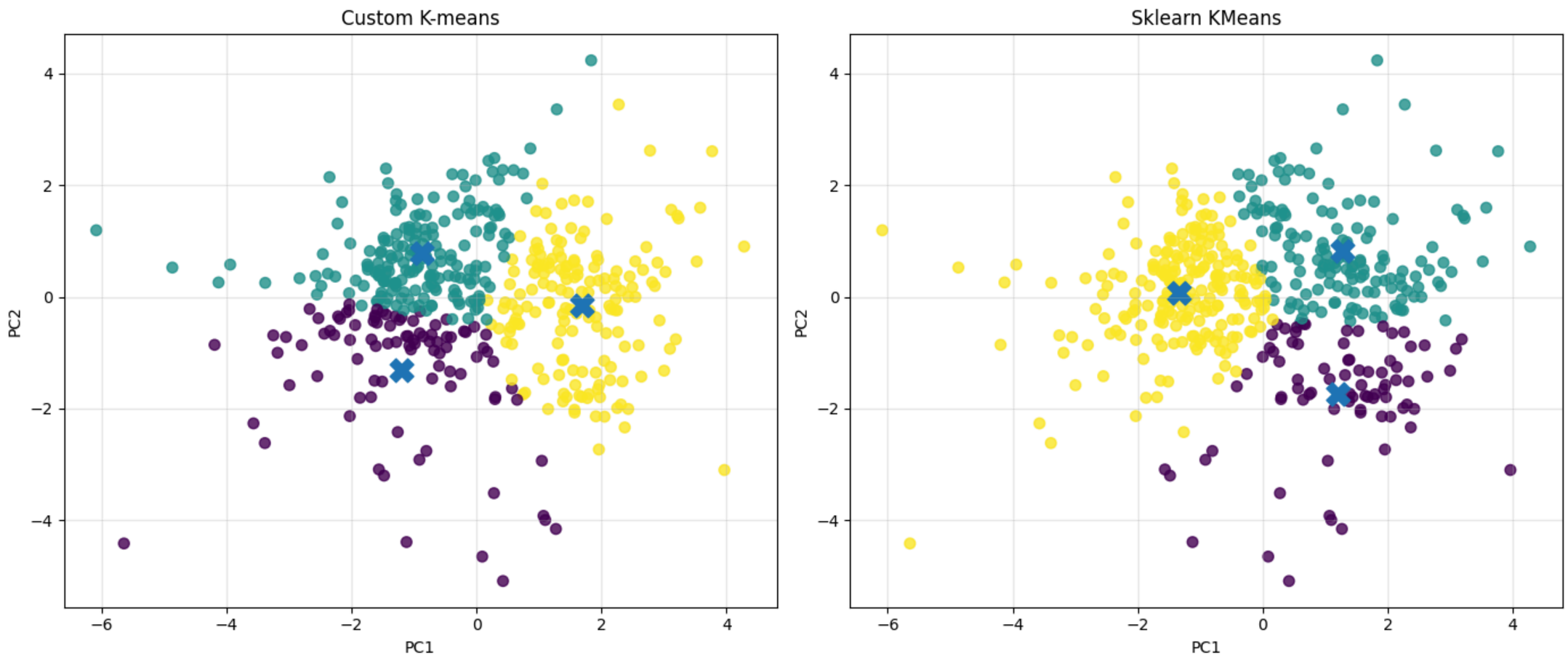
Centroid của Custom K-means (trên dữ liệu đã scale):

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
Custom Cluster 0	-0.5628	-0.7786	-0.6210	-0.6134	-0.6373	-1.0429
Custom Cluster 1	0.4707	-0.3582	-0.5024	0.6563	-0.5604	0.2474
Custom Cluster 2	-0.2102	0.8486	0.9215	-0.3908	0.9963	0.3158

Centroid của Sklearn KMeans (trên dữ liệu đã scale):

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
Sklearn Cluster 0	-1.1803	0.4774	0.6784	-1.1389	0.7833	-0.7011
Sklearn Cluster 1	0.4076	0.7987	0.7376	0.2958	0.6630	0.6824
Sklearn Cluster 2	0.1620	-0.7305	-0.7639	0.2236	-0.7518	-0.2076

Trực quan hóa cụm của hai mô hình trên PCA 2 chiều



→ Các cụm có xu hướng tách biệt theo hướng PC1, cho thấy đây là chiều quan trọng trong việc phân biệt khách hàng.

Nhận xét so sánh Custom K-means và Sklearn KMeans

Từ hai biểu đồ trực quan hóa bằng PCA, có thể nhận thấy:

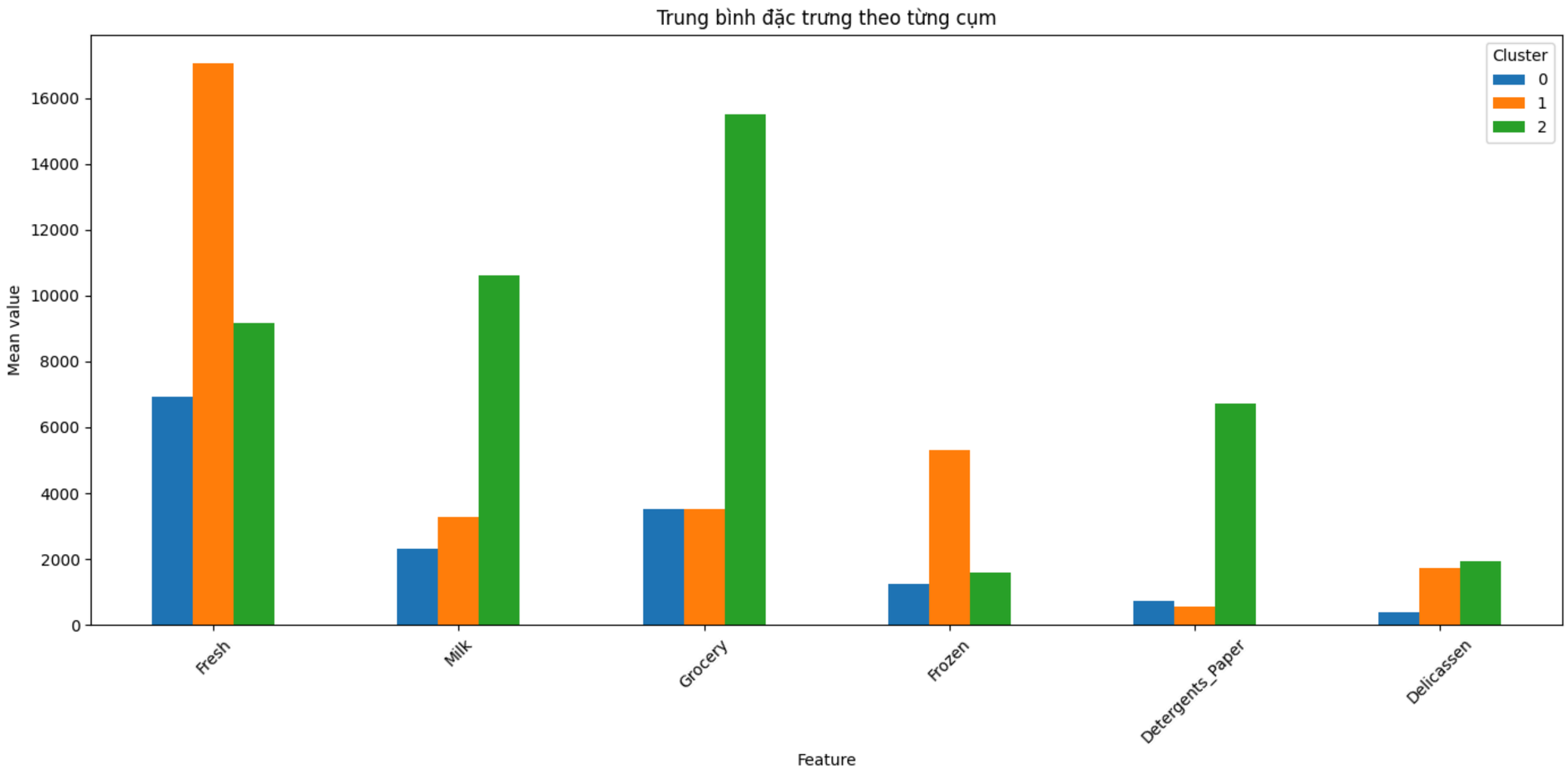
- Các cụm dữ liệu được phân chia khá tương đồng giữa hai mô hình.
- Vị trí các centroid (dấu X) gần như trùng nhau.
- Hình dạng và phân bố của các cụm không có sự khác biệt đáng kể.

Điều này cho thấy mô hình K-means tự cài đặt đã hoạt động đúng và cho kết quả tương đương với thư viện sklearn.

Sự khác biệt nhỏ (nếu có) là do:
- Khởi tạo centroid ban đầu khác nhau
- Sklearn sử dụng nhiều lần khởi tạo (n_init) để tối ưu kết quả

→ Kết luận: Custom K-means là chính xác và đáng tin cậy.

Trực quan hóa trung bình đặc trưng theo cụm



→ K-Means đã phân nhóm khách hàng thành các cụm có hành vi tiêu dùng khác nhau, hỗ trợ hiệu quả cho phân khúc và ra quyết định kinh doanh.

Kết Luận

1. Kết quả đạt được
   - Xây dựng thành công Custom K-Means
   - Hiểu rõ cơ chế:
     - Assignment (gán cụm)
     - Update (cập nhật centroid)
   - Kết quả tương đương với Sklearn KMeans
2. Nhận xét chính
   - K-Means phân cụm hiệu quả dựa trên hành vi chi tiêu
   - Các cụm thể hiện rõ sự khác biệt khách hàng
   - Kết quả phụ thuộc vào:
     - khởi tạo centroid
     - số cụm K
3. Ứng dụng thực tế
   - Phân khúc khách hàng
   - Marketing cá nhân hóa
   - Phân tích hành vi tiêu dùng
4. Hạn chế
   - Cần chọn trước K
   - Nhạy với dữ liệu và khởi tạo
   - Không tốt với dữ liệu phức tạp

Thanks For Watching